# Application Profiling




© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



# Profiling

- Profiling is the act of gathering data from a program execution in order to analyze them and then optimize or fix performance issues.
- Profiling is achieved by using programs that insert instrumentation in the code or leverage kernel/userspace mechanisms.
  - Profiling function calls and count of calls allow to optimize performance.
  - Profiling processor usage allows to optimize performance and reduce power usage.
  - Profiling memory usage allows to optimize memory consumption.
- After profiling, the data set must be analyzed to identify potential improvements (and not the reverse!).



# Performance issues

*"Premature optimization is the root of all evil", Donald Knuth*

- Profiling is often useful to identify and fix performance issues.
- Performance can be affected by memory usage, IOs overload, or CPU usage.
- Gathering profiling data before trying to fix performance issues is needed to do the correct choices.
- Profiling is often guided by a first coarse-grained analysis using some classic tools.
- Once the class of problems has been identified, a fine-grained profiling analysis can be done.



# Profiling metrics



- Multiple tools allow to profile various metrics.
- Memory usage with *Massif*, `heaptrack` or memusage.
- Function calls using *perf* and callgrind.
- CPU hardware usage (Cache, MMU, etc) using *perf*.
- Profiling data can include both the user space application and kernel.





# Memory profiling



# Memory profiling

- Profiling memory usage (heap/stack) in an application is useful for optimization.
- Allocating too much memory can lead to system memory exhaustion.
- Allocating/freeing memory too often can lead to the kernel spending a considerable amount of time in `clear_page()`.
  - The kernel clears pages before giving them to processes to avoid data leakage.
- Reducing application memory footprint can allow optimizing cache usage as well as page miss.



# Massif usage

- *Massif* is a tool provided by *valgrind* which allows to profile heap usage during the program execution (user-space only).
- Works by making snapshots of allocations.

```
$ valgrind --tool=massif --time-unit=B program
```

- Once executed, a *massif.out.* file will be generated in the current directory
- `ms_print` tool can then be used to display a graph of heap allocation

```
$ ms_print massif.out.275099
```

- `#`: Peak allocation
- `@`: Detailed snapshot (count can be adjusted thanks to `--detailed-freq`)



# Massif report (1/2)

```
    KB
547.0^                                                 # :: : :@ : :: :
     |                                                @:#:::::::@:::::::@
     |                                              ::@:#:::::::@:::::::@::
     |                                            :::::@:#:::::::@:::::::@:::::
     |                                         :::::::@:#:::::::@:::::::@:::::::
     |                                         :::::::@:#:::::::@:::::::@:::::::
     |                                         :::::::@:#:::::::@:::::::@:::::::
     |                                         :::::::@:#:::::::@:::::::@:::::::
     |                                 @@@@@@@@:::::::@:#:::::::@:::::::@:::::::
     |                                 @       :::::::@:#:::::::@:::::::@:::::::
     |                                 @       :::::::@:#:::::::@:::::::@:::::::
     |                           :::::::@       :::::::@:#:::::::@:::::::@:::::::
     |                           :     @       :::::::@:#:::::::@:::::::@:::::::
     |                     :::::::     @       :::::::@:#:::::::@:::::::@:::::::
     |                     :     :     @       :::::::@:#:::::::@:::::::@:::::::
     |               ::::::      :     @       :::::::@:#:::::::@:::::::@:::::::
     |               :     :     :     @       :::::::@:#:::::::@:::::::@:::::::
     |           :::::     :     :     @       :::::::@:#:::::::@:::::::@:::::::
     |      ::::    :     :     :     @       :::::::@:#:::::::@:::::::@:::::::
     |  ::::  :    :     :     :     @       :::::::@:#:::::::@:::::::@:::::::
   0 +-----------------------
```



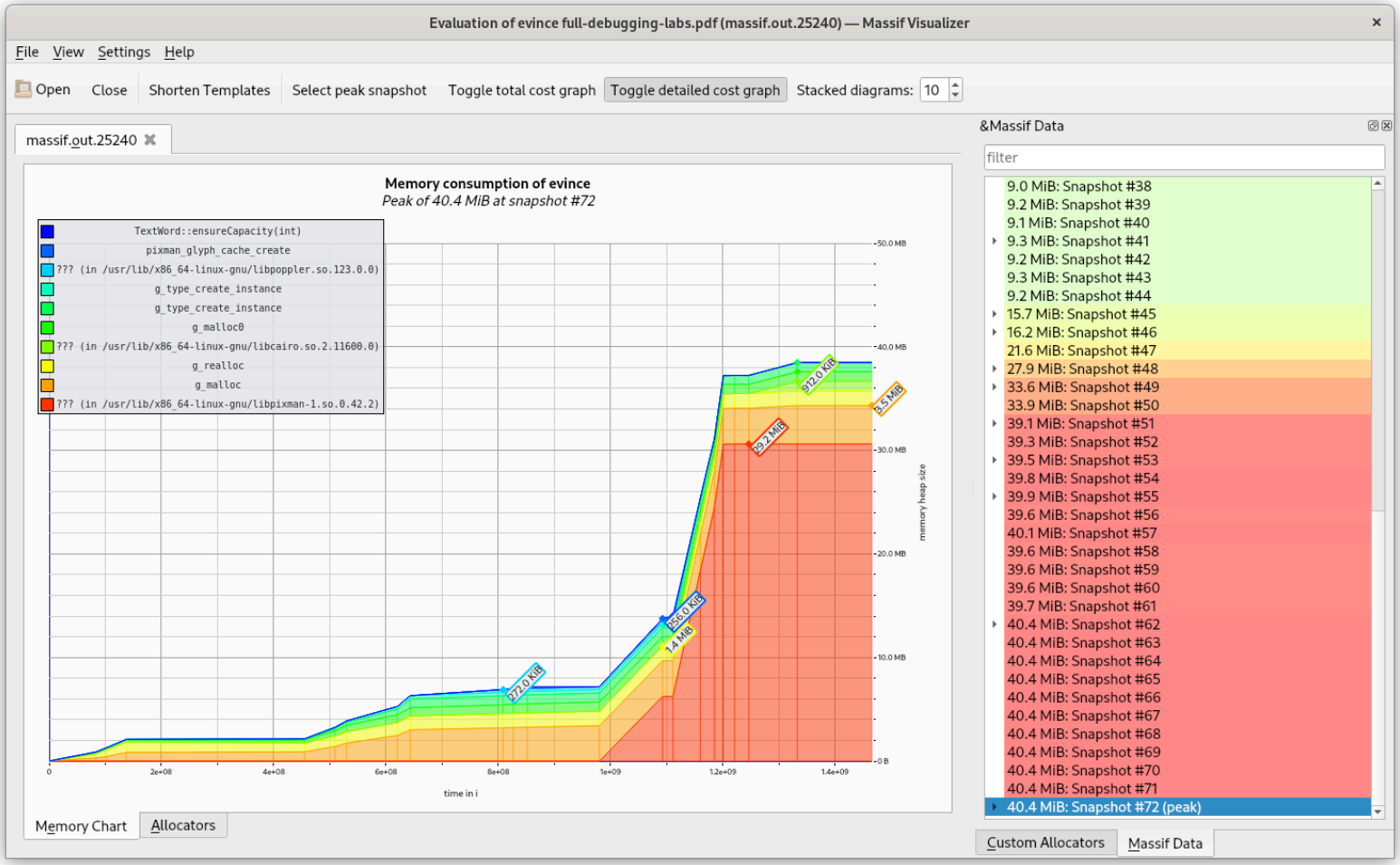
# Massif report (2/2)

```
--------------------------------------------------------------------------------
  n        time(B)         total(B)   useful-heap(B) extra-heap(B)    stacks(B)
--------------------------------------------------------------------------------
  0              0                0              0             0            0
  1              8                8              0             8            0
  2         10,016           10,016         10,000            16            0
  3         30,024           30,024         30,000            24            0
  4         60,032           60,032         60,000            32            0
  5        100,040          100,040        100,000            40            0
  6        150,048          150,048        150,000            48            0
  7        210,056          210,056        210,000            56            0
  8        280,064          280,064        280,000            64            0
  9        360,072          360,072        360,000            72            0
99.98% (360,000B) (heap allocation functions) malloc/new/new[], --alloc-fns, etc.
->99.98% (360,000B) 0x1088BF: allocate_large (heap_profile.c:46)
  ->99.98% (360,000B) 0x108947: allocate (heap_profile.c:53)
    ->99.98% (360,000B) 0x108973: main (heap_profile.c:61)

--------------------------------------------------------------------------------
  n        time(B)         total(B)   useful-heap(B) extra-heap(B)    stacks(B)
--------------------------------------------------------------------------------
 10        450,080          450,080         450,000            80            0
 11        460,088          460,088         460,000            88            0
 12        470,096          470,096         470,000            96            0
[...]
```



# massif-visualizer - Visualizing massif profiling data





# heaptrack usage

- *heaptrack* is a heap memory profiler for Linux.
  - Works with `LD_PRELOAD` library.
- Finer tracking than with Massif and visualizing tool is more advanced.
  - Each allocation is associated to a stacktrace.
  - Allows finding memory leaks, allocation hotspots and temporary allocations.
- Results can be seen using GUI (`heaptrack_gui`) or CLI tool (`heaptrack_print`).
- https://apps.kde.org/heaptrack/

```
$ heaptrack program
```

- This will generate a `heaptrack.<process_name>.<pid>.zst` file that can be analyzed using `heaptrack_gui` on another computer.



# heaptrack_gui - Visualizing heaptrack profiling data

Heaptrack - heaptrack.heaptrack_gui.12789.gz — Heaptrack GUI

Summary | Bottom-Up | Caller / Callee | Top-Down | Flame Graph | Consumed | Allocations | Temporary Allocations | Allocated | Sizes

**debuggee**:
heaptrack_gui heaptrack.kdevelop.17117
**total runtime**:
8.599s
**total system memory**:
11.4 GiB

**calls to allocation functions**:
2905522 (337890/s)
**temporary allocations**:
47978 (1.65%, 5579/s)
**bytes allocated in total** (ignoring deallocations):
341.46 MiB (39.7 MiB/s)

**peak heap memory consumption**:
267.3 MiB after 6.182s
**peak RSS** (including heaptrack overhead):
97.3 MiB
**total memory leaked**:
373.1 KiB

**Highest Memory Peaks**

| Peak | Location |
|---|---|
| 249.4 MiB | QArrayData::... |
| 9.0 MiB | void std::vect... |
| 6.0 MiB | void std::vect... |
| 5.0 MiB | std::__detail::... |
| 4.5 MiB | handleAlloca... |
| 3.0 MiB | void std::vect... |
| 1.4 MiB | QImageData:... |
| 1.0 MiB | 0x7fbdacc0d... |
| 1,020.1 KiB | run in lambd... |
| 768.0 KiB | Accumulated... |
| 768.0 KiB | void std::vect... |
| 768.0 KiB | void std::vect... |
| 645.2 KiB | QHashData::... |
| 611.4 KiB | QHashData::r... |
| 512.0 KiB | Accumulated... |
| 390.7 KiB | 0x7fbdd0250... |
| 390.7 KiB | 0x7fbdd0250... |
| 384.0 KiB | Accumulated... |
| 365.0 KiB | QRasterPaint... |
| 350.0 KiB | 0x7fbdd4100... |
| 344.1 KiB | QTextEngine:... |
| 326.3 KiB | void std::__cx... |
| 318.1 KiB | 0x7fbdd4100... |
| 308.0 KiB | 0x7fbdd4100... |
| 288.4 KiB | __alloc_dir in ... |

**Largest Memory Leaks**

| Leaked | Location |
|---|---|
| 71.0 KiB | _GLOBAL__su... |
| 69.2 KiB | 0x7fbdd0a9f... |
| 31.9 KiB | 0x7fbdc2884... |
| 27.3 KiB | _dl_new_obje... |
| 17.0 KiB | g_malloc in ?... |
| 16.0 KiB | 0x7fbdd072f... |
| 15.2 KiB | g_malloc0 in ... |
| 13.7 KiB | 0x7fbdc289c... |
| 13.1 KiB | __GI___strdup... |
| 7.0 KiB | 0x7fbdc289c... |
| 6.7 KiB | 0x7fbdc289c... |
| 6.5 KiB | _nl_intern_lo... |
| 5.6 KiB | _XlcCreateDe... |
| 5.6 KiB | 0x7fbdcd4fc... |
| 5.0 KiB | g_realloc in ?... |
| 4.7 KiB | _dl_check_m... |
| 4.3 KiB | FcConfigAdd... |
| 4.2 KiB | _XlcCreateLo... |
| 4.1 KiB | _XrmInternal... |
| 4.0 KiB | 0x7fbdd6ce4... |
| 3.6 KiB | read_alias_fil... |
| 3.6 KiB | 0x7fbdd074a... |
| 2.8 KiB | _dl_map_obje... |
| 2.6 KiB | _XlcAddCT in... |
| 2.0 KiB | FcFontSetAd... |

**Most Memory Allocations**

| Allocations | Location |
|---|---|
| 2694490 | QArrayData::... |
| 21754 | run in lambd... |
| 19350 | QHashData::... |
| 11225 | QListData::de... |
| 7797 | QImageData:... |
| 7241 | QRasterPaint... |
| 6209 | QBrush::init(... |
| 6086 | __GI___strdup... |
| 5816 | QRegion::cop... |
| 5641 | QByteArray::r... |
| 5167 | QMapDataBa... |
| 4806 | void std::__cx... |
| 4060 | 0x7fbdd0250... |
| 4060 | 0x7fbdd0250... |
| 4060 | hb_buffer_cre... |
| 3896 | 0x7fbdacc0d... |
| 3764 | QRegion::QR... |
| 3599 | QRawFont::Q... |
| 3547 | 0x7fbdd39a8... |
| 3379 | 0x7fbdc2897... |
| 2958 | QListData::re... |
| 2764 | QList<QVaria... |
| 2365 | QRasterPaint... |
| 2190 | 0x7fbdd3941... |
| 1921 | QTextEngine:... |

**Most Temporary Allocations**

| Temporary | Location |
|---|---|
| 21840 | QArrayData::... |
| 7714 | QImageData:... |
| 3990 | 0x7fbdd0250... |
| 2774 | QByteArray::... |
| 1990 | QRasterPaint... |
| 1975 | __GI___strdup... |
| 920 | QObject::QO... |
| 905 | RulerAttribut... |
| 784 | QListData::d... |
| 610 | QBrush::init(... |
| 482 | 0x7fbdccc7e... |
| 473 | QListData::re... |
| 400 | QRawFont::Q... |
| 332 | 0x7fbdd410c... |
| 290 | QLocale::set... |
| 248 | QString::reall... |
| 231 | QTimerInfoLi... |
| 218 | QBezier::add... |
| 183 | QMapDataBa... |
| 175 | 0x7fbdccc7d... |
| 168 | FrameAttribu... |
| 162 | QPaintEngine... |
| 93 | 0x7fbdccc7e... |
| 84 | QList<KChart... |
| 82 | QFileInfo::QFi... |

**Most Memory Allocated**

| Allocated | Location |
|---|---|
| 264.7 MiB | QArrayData::... |
| 10.5 MiB | void std::vect... |
| 7.0 MiB | void std::vect... |
| 6.7 MiB | QRasterPaint... |
| 6.0 MiB | handleAlloca... |
| 5.0 MiB | std::__detail::... |
| 4.0 MiB | void std::vect... |
| 3.9 MiB | 0x7fbdd0250... |
| 3.9 MiB | 0x7fbdd0250... |
| 2.7 MiB | QTextEngine:... |
| 1.7 MiB | QImageData:... |
| 1.2 MiB | hb_buffer_cr... |
| 1.0 MiB | 0x7fbdacc0d... |
| 1,024.0 KiB | void std::vect... |
| 1,020.1 KiB | run in lambd... |
| 974.6 KiB | QImageData:... |
| 801.2 KiB | __alloc_dir in ... |
| 775.4 KiB | 0x7fbdc289b... |
| 768.0 KiB | Accumulated... |
| 768.0 KiB | void std::vect... |
| 697.6 KiB | QBrush::init(... |
| 672.5 KiB | QRasterPaint... |
| 671.2 KiB | QListData::re... |
| 650.1 KiB | QHashData::... |
| 648.4 KiB | QHashData::r... |

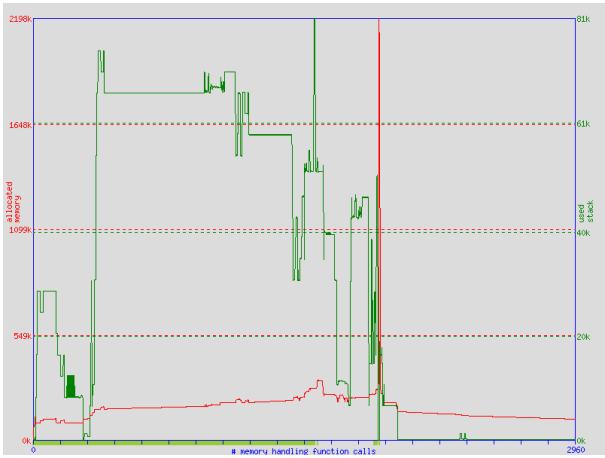
# heaptrack_gui - Flamegraph view





# memusage

- memusage is a program that leverages `libmemusage.so` to profile memory usage (man 1 memusage) (user-space only).
- Can profile heap, stack and also mmap memory usage.
- Profiling information can be shown on the console, logged to a file for post-treatment or visualized in a PNG file.
- Lightweight solution compared to valgrind *Massif* tool since it uses the `LD_PRELOAD` mechanism.





# memusage usage

```
$ memusage convert foo.png foo.jpg
Memory usage summary: heap total: 2635857, heap peak: 2250856, stack peak: 83696
         total calls   total memory   failed calls
 malloc|       1496        2623648              0
realloc|          6           3744              0  (nomove:0, dec:0, free:0)
 calloc|         16           8465              0
   free|       1480        2521334
Histogram for block sizes:
    0-15            329  21% ================================================
   16-31            239  15% ===================================
   32-47            287  18% ==========================================
   48-63            321  21% ================================================
   64-79             43   2% ======
   80-95            141   9% ====================
  ...
21424-21439           1  <1%
32768-32783           1  <1%
32816-32831           1  <1%
   large              3  <1%
```





# Execution profiling



# Execution profiling

- In order to optimize a program, one may have to understand what hardware resources are used.
- Many hardware elements can have an impact on the program execution:
  - CPU cache performance can be degraded by an application without memory spatial locality.
  - Page miss due to using too much memory without spatial locality.
  - Alignment faults when doing misaligned accesses.



# Using *perf stat*

- `perf stat` allows to profile an application by gathering performance counters.
  - performance counters are special-purpose registers use to count specific hardware events: CPU cycles, retired instructions, cache misses, branch prediction misses, etc
  - Using performance counters might require *root* permissions. This can be modified using `# echo -1 > /proc/sys/kernel/perf_event_paranoid`
- The number of performance counters that are present on the hardware are often limited (eg: 4 performance counters per core on ARM Cortex A7)
  - See perf wiki for more information.



# perf stat example (1/2)

```
$ perf stat convert foo.png foo.jpg

Performance counter stats for 'convert foo.png foo.jpg':

          45,52 msec task-clock                #    1,333 CPUs utilized
              4      context-switches          #   87,874 /sec
              0      cpu-migrations            #    0,000 /sec
          1 672      page-faults               #   36,731 K/sec
    146 154 800      cycles                    #    3,211 GHz                      (81,16%)
      6 984 741      stalled-cycles-frontend   #    4,78% frontend cycles idle     (91,21%)
     81 002 469      stalled-cycles-backend    #   55,42% backend cycles idle      (91,36%)
    222 687 505      instructions              #    1,52  insn per cycle
                                               #    0,36  stalled cycles per insn  (91,21%)
     37 776 174      branches                  #  829,884 M/sec                    (74,51%)
        567 408      branch-misses             #    1,50% of all branches          (70,62%)

    0,034156819 seconds time elapsed

    0,041509000 seconds user
    0,004612000 seconds sys
```

*NOTE: the percentage displayed at the end denotes the actual amount of time during which the kernel measured the event. It means that we have requested more events than available performance counters, so the result is actually a scaled approximation (due to multiplexing). To fix the measure imprecision, reduce the number of observed events / run multiple* `perf stat` *records to collect all the expected events.*



# perf stat example (2/2)

- List all events:

```
$ perf list
  List of pre-defined events (to be used in -e):

  branch-instructions OR branches                    [Hardware event]
  branch-misses                                      [Hardware event]
  cache-misses                                       [Hardware event]
  cache-references                                   [Hardware event]
  ...
```

- Count *L1-dcache-load-misses* and *branch-load-misses* events for a specific command

```
$ perf stat -e L1-dcache-load-misses,branch-load-misses cat /etc/fstab
...
Performance counter stats for 'cat /etc/fstab':

23 418      L1-dcache-load-misses
 7 192      branch-load-misses
...
```



# Cachegrind

- *Cachegrind* is a tool provided by *valgrind* for profiling program interactions with the instruction and data cache hierarchy.
  - *Cachegrind* also profiles branch prediction success.
- Simulate a machine with independent `I$` and `D$` backed with a unified L2 cache.
- Really helpful to detect cache usage problems (too many misses, etc).

```
$ valgrind --tool=cachegrind --cache-sim=yes ./my_program
```

- It generates a `cachegrind.out.<pid>` file containing the measures
- `cg_annotate` is a CLI tool used to visualize cachegrind simulation results.
- It also has a `--diff` option to allow comparing two measures files



# Kcachegrind - Visualizing Cachegrind profiling data

cachegrind.out.27592 [./a.out ../tux.png out.png] — KCachegrind

Fichier Affichage Aller Configuration Aide

Ouvrir | Précédent | Suivant | Remonter | Relatif | Détection de cycles | Relatif au parent | Raccourcir des modèles | Cycle Estimation

&Profilage à plat

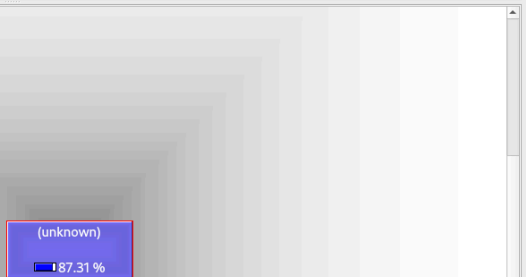
Search: Search Query (No Grouping)

| Inc | Self | Cal | Function | Location |
|---|---|---|---|---|
| ... | 87.31 | (0) | (unknown) | (unknown) |
| ... | 5.20 | (0) | __memcpy_avx_unalign... | memmove-vec-unaligned-erms.S |
| ... | 4.11 | (0) | adler32_z | (unknown) |
| ... | 3.20 | (0) | swap_colors | png.c |
| ... | 0.07 | (0) | inflate | (unknown) |
| ... | 0.05 | (0) | crc32_z | (unknown) |
| ... | 0.01 | (0) | _int_malloc | malloc.c |
| ... | 0.01 | (0) | deflate | (unknown) |
| ... | 0.01 | (0) | png_read_row | (unknown) |
| ... | 0.01 | (0) | _int_free | malloc.c |
| ... | 0.01 | (0) | __memset_avx2_unalig... | memset-vec-unaligned-erms.S |
| ... | 0.01 | (0) | png_write_row | (unknown) |
| ... | 0.00 | (0) | do_lookup_x | dl-lookup.c |
| ... | 0.00 | (0) | sysmalloc | malloc.c |
| ... | 0.00 | (0) | malloc | malloc.c |
| ... | 0.00 | (0) | _dl_relocate_object | dl-reloc.c |
| ... | 0.00 | (0) | read_png_file | png.c |
| ... | 0.00 | (0) | png_write_image | (unknown) |
| ... | 0.00 | (0) | write_png_file | png.c |
| ... | 0.00 | (0) | unlink_chunk.constprop.0 | malloc.c |
| ... | 0.00 | (0) | free | malloc.c |
| ... | 0.00 | (0) | _dl_lookup_symbol_x | dl-new-hash.h |
| ... | 0.00 | (0) | png_read_image | (unknown) |
| ... | 0.00 | (0) | strcmp | strcmp-sse2.S |
| ... | 0.00 | (0) | __GI___tunables_init | dl-tunables.c |
| ... | 0.00 | (0) | systrim.constprop.0 | malloc.c |
| ... | 0.00 | (0) | _dl_relocate_object | do-rel.h |
| ... | 0.00 | (0) | _dl_lookup_symbol_x | dl-lookup.c |
| ... | 0.00 | (0) | check_match | dl-lookup.c |
| ... | 0.00 | (0) | _dl_relocate_object | dl-machine.h |
| ... | 0.00 | (0) | _dl_check_map_versions | dl-version.c |
| ... | 0.00 | (0) | _dl_map_object_from_fd | dl-load.c |
| ... | 0.00 | (0) | _IO_file_xsgetn | fileops.c |
| ... | 0.00 | (0) | _dl_fixup | dl-runtime.c |
| ... | 0.00 | (0) | png_get_rowbytes | (unknown) |
| ... | 0.00 | (0) | dl_main | rtld.c |
| ... | 0.00 | (0) | fread | iofread.c |
| ... | 0.00 | (0) | adler32 | (unknown) |
| ... | 0.00 | (0) | _dl_map_object_from_fd | get-dynamic-info.h |
| ... | 0.00 | (0) | fwrite | iofwrite.c |

(unknown)

Types | Callers | All Callers | Callee Map | Source Code

| Event Type | Incl. | Self | Short | Formula |
|---|---|---|---|---|
| Instruction Fetch | 89.19 | 89.19 | Ir | |
| L1 Instr. Fetch Miss | 27.61 | 27.61 | I1mr | |
| LL Instr. Fetch Miss | 27.99 | 27.99 | ILmr | |
| Data Read Access | 70.99 | 70.99 | Dr | |
| L1 Data Read Miss | 73.90 | 73.90 | D1mr | |
| LL Data Read Miss | 1.58 | 1.58 | DLmr | |
| Data Write Access | 59.55 | 59.55 | Dw | |
| L1 Data Write Miss | 42.71 | 42.71 | D1mw | |
| LL Data Write Miss | 0.72 | 0.72 | DLmw | |
| L1 Miss Sum | 60.08 | 60.08 | L1m | = I1mr + D1mr + D1mw |
| Last-level Miss Sum | 0.95 | 0.95 | LLm | = ILmr + DLmr + DLmw |
| Cycle Estimation | 87.31 | 87.31 | CEst | = Ir + 10 L1m + 100 LLm |




Parts | Callees | Call Graph | All Callees | Caller Map | Machine Code

cachegrind.out.27592 [1] - Total (Cycle Estimation) Coût total : 5 781 194 895



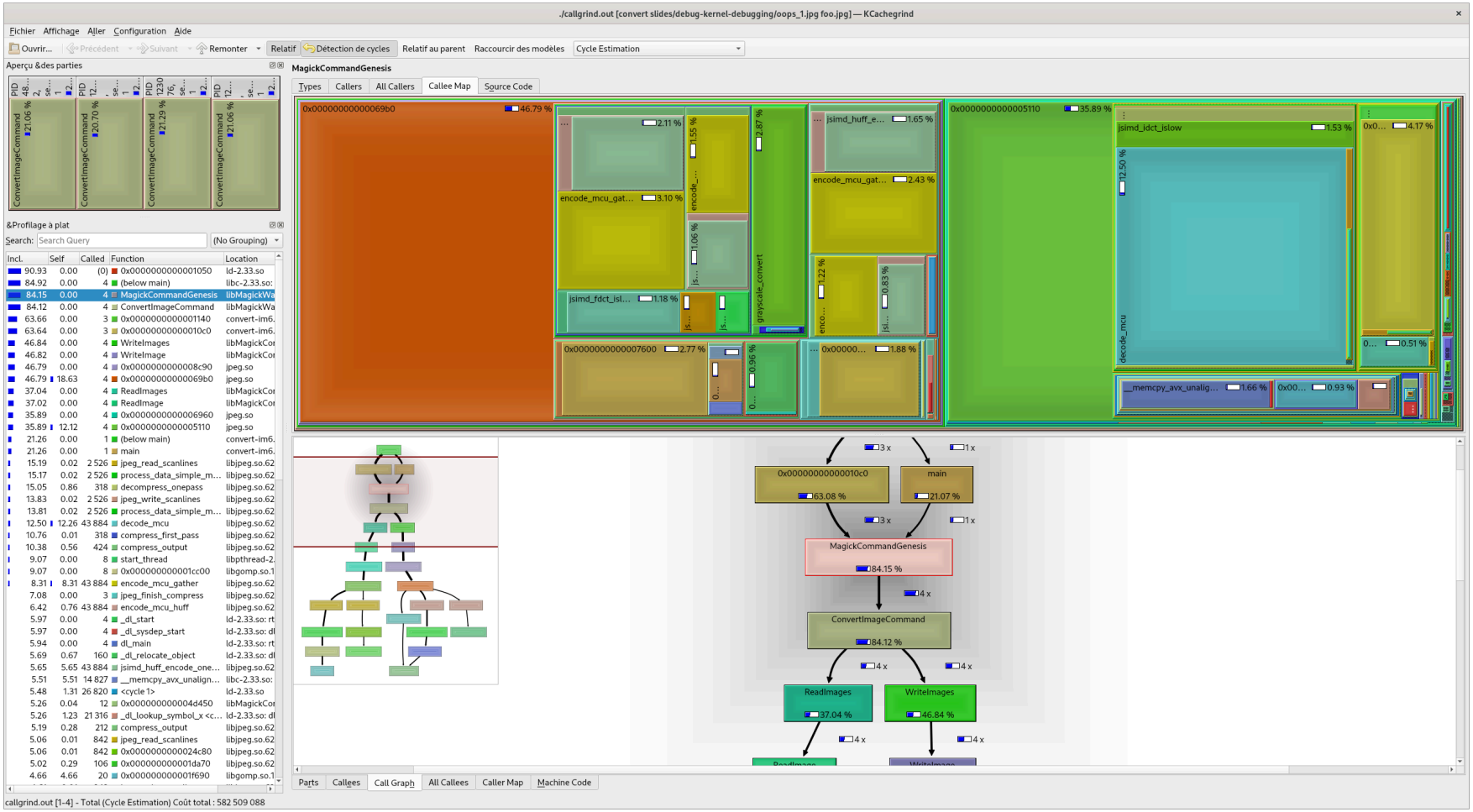
# Callgrind

- Provided by *valgrind* and allowing to profile an application call graph (user-space only).
- Collects the number of instructions executed during your program execution and associate these data with the source lines
- Records the call relationship between functions and their call count.

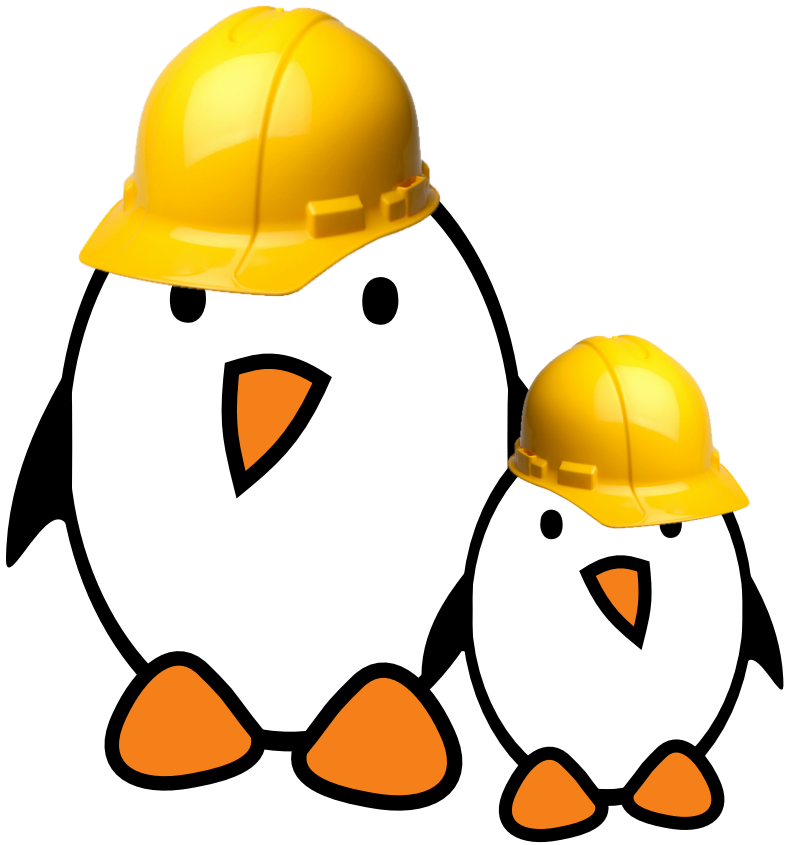
```
$ valgrind --tool=callgrind ./my_program
```

- `callgrind_annotate` is a CLI tool used to visualize callgrind simulation results.
- Kcachegrind can visualize *callgrind* results too.
- The cache simulation (done using cachegrind) has some accuracy shortcomings (See Cachegrind accuracy)

# Kcachegrind - Visualizing Callgrind profiling data



# Practical lab - Profiling applications



Profiling an application using various tools

- Profiling application heap using *Massif*.
- Profiling an application with *Cachegrind*, *Callgrind* and *KCachegrind*.
- Analyzing application performance with *perf*.